

Cơ sở dữ liệu thực hành

DML (tiếp)

Lê Thành Văn

Khoa Hệ thống Thông tin Quản lý

HUTECH

1. SELECT

2. JOIN

3. GROUP BY

4. Bài tập

SELECT

Câu lệnh **SELECT** được sử dụng để truy vấn dữ liệu từ một hoặc nhiều bảng trong cơ sở dữ liệu.

Trong phần này, chúng ta sẽ sử dụng cơ sở dữ liệu sau:

```
CREATE TABLE regions (  
    region_id INT PRIMARY KEY,  
    region_name VARCHAR(100));  
  
CREATE TABLE provinces (  
    province_id INT PRIMARY KEY,  
    region_id INT REFERENCES regions (region_id),  
    province_name VARCHAR(50));  
  
CREATE TABLE communes (  
    commune_id INT PRIMARY KEY,  
    province_id INT REFERENCES provinces (province_id),  
    commune_name VARCHAR(50));
```

Truy vấn không điều kiện từ một bảng

Cú pháp:

```
-- Truy vấn toàn bộ cột
SELECT
    *
    FROM tên_bảng;
-- Truy vấn một số cột
SELECT
    cột_1, cột_2, ..., cột_n
    FROM tên_bảng;
```

Có thể đổi tên cột trong kết quả trả về để dễ đọc hơn

```
-- Từ khóa AS có thể ghi hoặc không
SELECT
    cột_1 AS "tên_mới_1",
    cột_2 AS "tên_mới_2",
    ...,
    cột_n AS "tên_mới_n"
FROM tên_bảng
```

Không nên sử dụng **SELECT *** để xem tên cột của bảng, nếu muốn, có thể dùng giao diện hoặc \d.

```
-- Truy vấn toàn bộ tên tỉnh
SELECT
    province_name AS "Tỉnh/Thành phố"
FROM provinces
```

Để truy vấn có điều kiện, ta sử dụng mệnh đề **WHERE**:

```
SELECT  
    ...  
FROM tên_bảng  
WHERE điều_kiện;
```

`điều_kiện` trong câu truy vấn có thể là điều kiện đơn hoặc điều kiện ghép.
Điều kiện đơn thường có dạng `tên_cột toán_tử biểu_thức`.

Một số `toán_tử` thường sử dụng:

- `=, <>, >, ≥, <, ≤`: phép toán so sánh.
- `IS DISTINCT FROM/IS NOT DISTINCT FROM`: so sánh bằng/so sánh khác nhưng an toàn với `NULL`.
- `LIKE`: khớp chuỗi đơn giản.
- `IN/NOT IN`: kiểm tra giá trị có nằm trong một danh sách.
- `IS NULL/IS NOT NULL`: kiểm tra trống/không trống.
- `BETWEEN ... AND ...`: kiểm tra nằm trong khoảng.

Khi sử dụng toán tử so sánh, cần lưu ý rằng kết quả của phép so sánh mà có ít nhất một trong hai bên là **NULL** sẽ là **NULL**. Do đó, nó sẽ không được trả về.

```
-- Các phép toán sau sẽ trả về NULL
SELECT 1 = NULL;
SELECT 1 > NULL;
SELECT NULL <> NULL;
-- So sánh an toàn với NULL
SELECT 1 IS DISTINCT FROM NULL; -- trả về TRUE
SELECT NULL IS NOT DISTINCT FROM NULL; -- trả về FALSE
```

LIKE là cách khớp chuỗi đơn giản của SQL, có hai ký tự đại diện:

- **%**: đại diện cho **một chuỗi** ký tự bất kỳ (bao gồm cả chuỗi rỗng).
- **_**: đại diện cho **đúng một** ký tự bất kỳ.

Lưu ý. **LIKE** khi khớp chuỗi sẽ phân biệt hoa thường.

```
-- Truy vấn tên các thành phố
SELECT
    province_name AS "Thành phố"
FROM provinces
WHERE province_name LIKE 'Thành phố%'
```

Dùng để kiểm tra giá trị thuộc/không thuộc một danh sách. Thường dùng chung với subquery.

```
-- Toán tử IN
SELECT 1 IN (1, 2);      -- TRUE
SELECT 1 IN (1, NULL);   -- TRUE
SELECT 1 IN (2, 3);      -- FALSE
SELECT 1 IN (2, NULL);   -- NULL
```

```
-- Toán tử NOT IN
SELECT 1 NOT IN (1, 2);      -- FALSE
SELECT 1 NOT IN (1, NULL);   -- FALSE
SELECT 1 NOT IN (2, 3);      -- TRUE
SELECT 1 NOT IN (2, NULL);   -- NULL
```

Dùng để kiểm tra giá trị nằm trong một khoảng, thường dùng với kiểu số hoặc thời gian.

Ví dụ 6

```
-- Toán tử NOT IN  
SELECT 1 BETWEEN 1 AND 2;      -- TRUE  
SELECT 3 BETWEEN 1 AND 2;      -- FALSE
```

Điều kiện ghép

Có thể ghép nhiều điều kiện với nhau bằng các toán tử logic **AND** và **OR**, ta có bảng chân trị khi ghép (T = **TRUE**, F = **FALSE**, N = **NULL**)

AND	T	F	N
T	T	F	N
F	F	F	F
N	N	F	N

OR	T	F	N
T	T	T	T
F	T	F	N
N	T	N	N

Để sắp xếp kết quả trả về, ta dùng từ khóa **ORDER BY**

```
SELECT
    ...
    FROM tên_bảng WHERE điều_kiện
    ORDER BY cột_1 ASC|DESC,
           ...,
           cột_n ASC|DESC
```

Trong đó, **ASC** là sắp xếp tăng dần, **DESC** là sắp xếp giảm dần.

Dùng để phân trang (paging) dữ liệu, thường dùng chung với **ORDER BY**.

```
SELECT
    ...
FROM tên_bảng WHERE điều_kiện
ORDER BY ...    -- Sắp xếp để đảm bảo lấy đúng dữ liệu
LIMIT ...      -- Lấy bao nhiêu dòng
OFFSET ...      -- Bỏ qua bao nhiêu dòng đầu tiên
```

Nếu một cột có nhiều giá trị giống nhau và chỉ muốn lấy danh sách các giá trị duy nhất, có thể dùng từ khóa **DISTINCT**.

```
SELECT
    DISTINCT tên_cột
FROM tên_bảng
WHERE điều_kiện;
```

Thứ tự thực thi của SELECT

PostgreSQL xử lý câu truy vấn theo thứ tự sau:

- 1 **FROM**: Xác định nguồn dữ liệu (Bảng).
- 2 **WHERE**: Lọc các dòng.
- 3 **GROUP BY**: Nhóm các dòng.
- 4 **HAVING**: Lọc các nhóm.
- 5 **SELECT**: Chọn các cột và biểu thức.
- 6 **DISTINCT**: Loại bỏ trùng lặp.
- 7 **ORDER BY**: Sắp xếp kết quả.
- 8 **LIMIT/OFFSET**: Giới hạn số dòng trả về.

Thứ tự thực thi trên cũng xác định cách PostgreSQL xử lý bí danh (alias), một bí danh ở phần có ưu tiên thấp hơn sẽ không thể được hiểu ở phần có ưu tiên cao hơn.

JOIN

Để truy vấn dữ liệu từ nhiều bảng, ta dùng đến kỹ thuật kết bảng.
Một số loại kết bảng thường dùng:

- **INNER JOIN/JOIN**: kết trong.
- **LEFT OUTER JOIN/LEFT JOIN**: kết ngoài trái.

```
SELECT ...  
    FROM bảng_trái  
        LEFT JOIN|JOIN bảng_phải ON điều_kiện_kết  
    WHERE ...
```

Trong đó, `điều_kiện_kết` thường được sử dụng là khóa ngoại `bảng_trái` bằng với khóa chính `bảng_phải`.

```
-- Truy vấn tên tỉnh và tên khu vực của tỉnh đó
SELECT p.province_name, r.region_name
       FROM provinces AS p -- đặt alias để viết gọn hơn
       JOIN regions AS r ON p.region_id = r.region_id;
```

```
-- Truy vấn tên phường, tên thành phố và tên khu vực
SELECT c.commune_name, p.province_name, r.region_name
FROM communes AS c
    JOIN provinces AS p ON c.province_id = p.province_id
    JOIN regions AS r ON p.region_id = r.region_id
WHERE c.commune_name LIKE 'Phường%'
    AND p.province_name LIKE 'Thành phố';
```

JOIN chỉ giữ lại những dòng mà `điều_kiện_kết` là đúng.

LEFT JOIN giữ nguyên tất cả những dòng ở bảng trái, những dòng mà `điều_kiện_kết` thì các cột của `bảng_phải` là **NULL**.

Bên cạnh hai kỹ thuật kết bảng ở trên, ta còn một số dạng kết bảng khác như:

- **RIGHT OUTER JOIN/RIGHT JOIN**: kết ngoài phải.
- **FULL OUTER JOIN**: kết ngoài đầy đủ.

GROUP BY

Kỹ thuật **GROUP BY** (gom nhóm) được sử dụng để nhóm các hàng có cùng giá trị trong các cột cụ thể thành các hàng tóm tắt.

Nó thường đi kèm với các hàm tổng hợp (aggregate function) như:

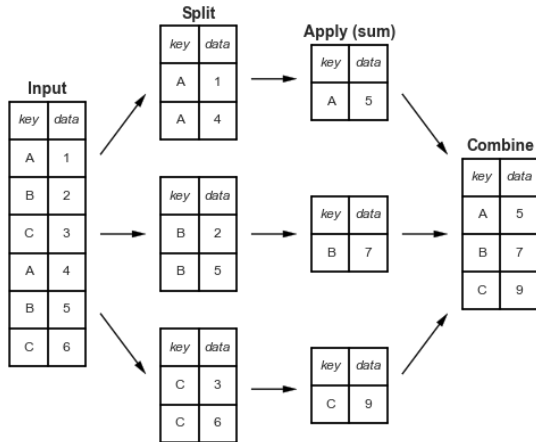
- **COUNT(*)**: đếm số dòng.
- **COUNT(cột)**: đếm số dòng khác **NULL**.
- **SUM(cột)**: trả về tổng.
- **AVG(cột)**: trả về trung bình cộng.
- **MIN(cột)/MAX(cột)**: trả về giá trị nhỏ nhất/lớn nhất.

SELECT

```
cột_gom_nhóm,  
hàm_tổng_hợp_1(cột_tổng_hợp_1),  
hàm_tổng_hợp_2(cột_tổng_hợp_2),  
...  
hàm_tổng_hợp_n(cột_tổng_hợp_n)  
FROM ... WHERE ...  
GROUP BY cột_gom_nhóm
```

- Tất cả các cột xuất hiện trong mệnh đề **SELECT** mà không nằm trong hàm tổng hợp bắt buộc phải có mặt trong mệnh đề **GROUP BY**.
- **GROUP BY** phải đứng sau **FROM** và **WHERE**, nhưng đứng trước **HAVING** và **ORDER BY**.

Cách hoạt động



```
-- Truy vấn mã khu vực và số lượng tỉnh mỗi khu vực
SELECT
    region_id, COUNT(province_id)
FROM provinces
GROUP BY region_id
```

Trong mệnh đề **GROUP BY**, để lọc kết quả của hàm tổng hợp sau khi gom nhóm, ta dùng từ khóa **HAVING**.

Lưu ý: **WHERE** dùng để lọc những dòng trước khi gom nhóm, **HAVING** để lọc kết quả sau khi gom nhóm.

```
-- Truy vấn mã tỉnh của những tỉnh có số phường lớn hơn 10
SELECT
    province_id, COUNT(commune_id) AS "Số phường"
FROM communes
WHERE commune_name LIKE 'Phường%' -- lọc ra phường
GROUP BY province_id
    HAVING COUNT(commune_id) > 10 -- lọc kết quả sau tổng hợp
```

Mệnh đề **FILTER** là một tính năng của PostgreSQL cho phép áp dụng các điều kiện lọc trực tiếp bên trong các hàm tổng hợp.

Cú pháp

```
hàm_tổng_hợp(...) FILTER(WHERE điều_kiện)
```

Khi bạn muốn tính toán nhiều chỉ số khác nhau từ cùng một bảng trong cùng một câu lệnh mà mỗi chỉ số lại có điều kiện lọc khác nhau.

```
-- Truy vấn mã tỉnh và số lượng phường, số lượng xã
SELECT
    province_id,
    COUNT(commune_id) FILTER(WHERE commune_name LIKE 'Phường%'),
    COUNT(commune_id) FILTER(WHERE commune_name LIKE 'Xã%')
FROM communes
GROUP BY province_id
```

Bài tập

Cho 3 bảng:

- Danh mục: `categories(category_id, category_name)`.
- Sản phẩm: `products(product_id, product_name, price, category_id)`.
- Đơn hàng: `orders(order_id, product_id, quantity, order_date)`.

Viết câu lệnh SQL cho các yêu cầu:

- 1 Lấy danh sách tất cả sản phẩm gồm: Tên sản phẩm, Giá và Tên danh mục tương ứng.
- 2 Đếm xem mỗi danh mục (`category_id`) có bao nhiêu sản phẩm.
- 3 Tính tổng doanh thu cho mỗi danh mục sản phẩm (Tên danh mục, Tổng tiền).
Tính tổng tiền bằng `SUM(products.price * orders.quantity)`.
- 4 Liệt kê tất cả các danh mục và số lượng sản phẩm trong mỗi danh mục đó.
Đảm bảo rằng những danh mục chưa có sản phẩm nào vẫn xuất hiện trong danh sách (với số lượng bằng 0).

-- câu 1

SELECT

p.product_name, p.price, c.category_name

FROM products p

JOIN categories c ON p.category_id = c.category_id;

-- câu 2

SELECT category_id, COUNT(product_id) AS total_products

FROM products

GROUP BY category_id;

-- câu 3

SELECT

 c.category_name,

 SUM(p.price * o.quantity) AS total_revenue

FROM categories c

 JOIN products p

 ON c.category_id = p.category_id

 JOIN orders o

 ON p.product_id = o.product_id

GROUP BY c.category_name;

```
-- câu 4
SELECT
    c.category_name,
    COUNT(p.product_id) AS product_count
FROM categories c
    LEFT JOIN products p
        ON c.category_id = p.category_id
GROUP BY c.category_name;
```